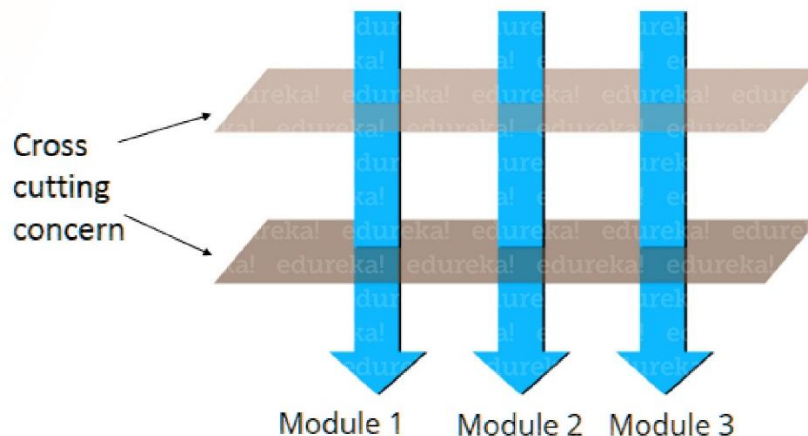


Spring Framework

AOP (Aspect-Oriented Programming) :-

What is AOP ?

- AOP separates cross-cutting concerns from main business logic.
- It helps avoid repetitive code like logging or security checks.
- Common logic is written in a single module called an Aspect.
- That Aspect can be applied to multiple methods or classes as needed.



Why AOP ?

- AOP helps to reduce code duplication by separating common logic.
- It improves modularity and makes code easier to manage.
- You can add or modify cross-cutting concerns (like logging, security, or transactions) without changing core code.
- It keeps the business logic clean and focused on its main purpose.

Core AOP Concepts :-

There are different core concepts of AOP are as follows :

1. Aspect

- An Aspect defines cross-cutting logic such as logging, security, or transaction management.
- It specifies what needs to be done and where it should be applied.

2. Join Point

- A Join Point is a specific execution point in the application, such as a method call or exception handling, where an Aspect can be applied.

3. Advice

- Advice defines the action taken by an Aspect at a particular Join Point.
- Types of Advice include:

1. **@Before:** Runs before the method execution.
2. **@After:** Runs after the method execution.
3. **@Around:** Runs both before and after the method execution.

4. Pointcut

- A Pointcut is an expression that determines which methods or classes an Advice should apply to.
- It identifies relevant Join Points.

5. Target Object

- The Target Object is the main object whose method is being advised by an Aspect.
- AOP dynamically adds extra behavior to this object without modifying its code.

6. Proxy

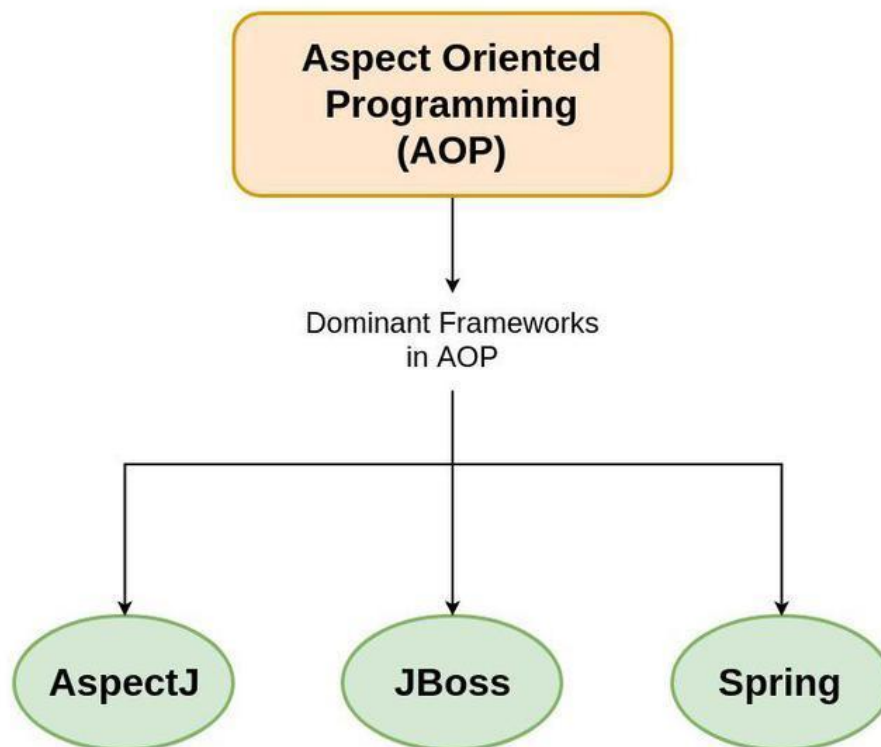
- A Proxy is the object created by the AOP framework that wraps the target object.
- It manages method calls and applies the specified Advice at the appropriate Join Points.

7. Weaving

- Weaving is the process of linking Aspects with target code to create an advised object.
- This can occur at compile-time, load-time, or runtime (in Spring AOP).



Dominant AOP Frameworks :-



The diagram lists three major frameworks that implement AOP concepts:

1. **AspectJ –**

- The most powerful and complete AOP framework for Java.
- It supports compile-time, load-time, and runtime weaving of aspects.

2. **JBoss AOP –**

- A framework integrated with the JBoss Application Server.
- It provides AOP support mainly for enterprise-level Java applications.

3. **Spring AOP –**

- A simplified and lightweight implementation of AOP in the Spring Framework.
- It focuses on runtime proxy-based AOP, making it easy to use within Spring applications.

Difference Between Spring AOP and Spring IOC :-

Parameter	Spring AOP	Spring IOC
Full form	Spring Aspect-Oriented Programming	Spring Inversion of Control
Main Concept	Handling cross-cutting concerns	Dependency injection and object lifecycle management.
Purpose	Logging, transaction management and security.	Managing object creation and injecting dependencies
Design Pattern	Proxy Pattern	Factory and Singleton pattern
Implementation	Using pointcuts, join points and advices.	Using BeanFactory or ApplicationContext.
Effects on code	Adds behavior without modifying actual code	Wires and manages object dependencies.
Configure	Annotation like @Aspect or via AOP namespace.	Java-based configurations or we can Configure XML file.

Steps to create AOP Example :-

Here are the steps to implement AOP:

1. Add Maven Dependencies

- Include the required Spring AOP and AspectJ dependencies in your pom.xml.
- These provide the libraries needed to enable AOP features in your project.

2. Create Aspect and Pointcut Expression

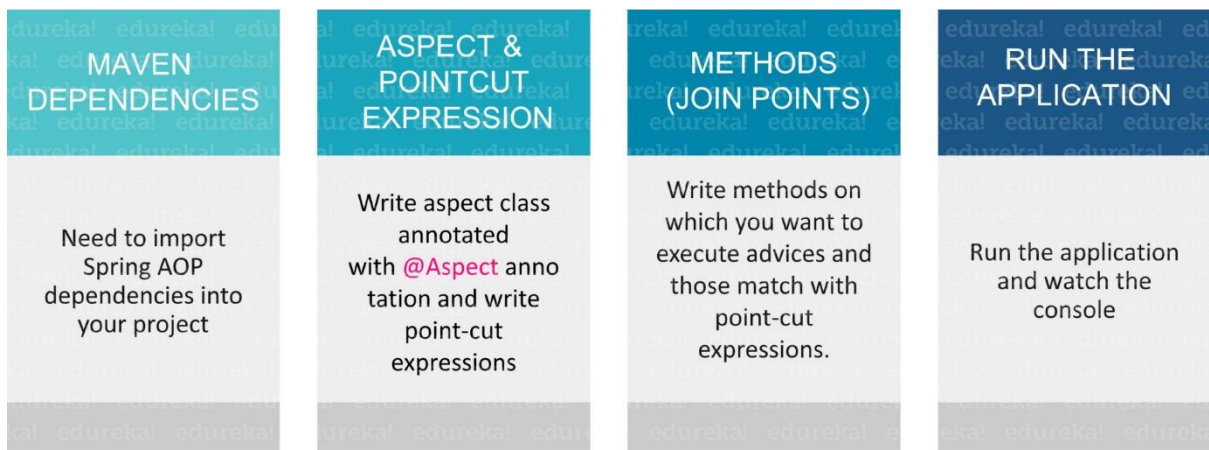
- Write a class annotated with `@Aspect`.
- Inside this class, define Pointcut expressions to specify where the AOP logic (Advice) should run.

3. Define Methods (Join Points)

- In your business logic class, write the methods you want to apply the aspect to.
- These methods will match the Pointcut expressions and trigger the Advice.

4. Run the Application

- Execute your Spring application.
- You'll see the Advice messages (like "Before" or "After" logs) printed in the console — showing AOP in action.



Implementation of AOP (Aspect-Oriented Programming) :-

Student.java –

```
package Demo;

public class Student {

    public void display() {
        System.out.println("I am display method");
    }
}
```

Logfile.java –

```
package Demo;

import org.aspectj.lang.annotation.After;
import org.aspectj.lang.annotation.Aspect;
import org.aspectj.lang.annotation.Before;

@Aspect
public class Logfile {

    @Before("execution(* Demo.Student.display(..))")
    public void beforeDisplay() {
        System.out.println("Before Display");
    }

    @After("execution(* Demo.Student.display(..))")
    public void afterDisplay() {
        System.out.println("After Display");
    }

}
```

applicationContext.java –

```
package Demo;

import org.aspectj.lang.annotation.After;
import org.aspectj.lang.annotation.Aspect;
import org.aspectj.lang.annotation.Before;

@Aspect
public class Logfile {
```

```

@Before("execution(* Demo.Student.display(..))")
public void beforeDisplay() {
    System.out.println("Before Display");
}

@After("execution(* Demo.Student.display(..))")
public void afterDisplay() {
    System.out.println("After Display");
}
}

```

Test.java –

```

package Demo;

import org.springframework.context.ApplicationContext;
import org.springframework.context.support.ClassPathXmlApplicationContext;

public class Test {

    public static void main(String[] args) {
        ApplicationContext con=new ClassPathXmlApplicationContext("applicationContext.xml");

        Student s=con.getBean(Student.class);
        s.display();
    }
}

```

Output :-

```

Before Display
I am display method
After Display

```